

OS Development

2. Introduction to OS development

Prof. Dr. Michael Schöttner

Dr. Fabian Ruhland

- Classification
- Compiling and Linking
- Boot process
- Debugging
- Summary

- First steps: How to bring your OS to the target hardware?
 - Compiling
 - Boot process
- Testing and Debugging: What to do in case the system is stuck?
 - „println()“ debugging
 - Emulation
 - Debugger
 - Remote debugger
 - Hardware support

- Classification
- **Compiling and Linking**
- Boot process
- Debugging
- Summary

Compiling – Hello, World?

```
fn main() {  
    println!("hello world");  
}
```

```
$ rustc hello.rs  
$ ./hello  
hello world
```

- Assumption:
 - The development environment runs on Linux, macOS or Windows
 - The target system is a x86_64 system
- Does this program also run directly on *naked* hardware?
- Can an operating system even be developed in a high-level language

- No dynamic linking available → All required libraries must be linked statically
 - Runtime environment `std` expects OS functions for threads, memory management, network, etc.
 - Cannot be used, as well as other libraires (*crates*) requiring `std`
 - But we can use the Rust `core` crate (and `alloc`, once we have implemented a heap allocator)
- All generated addresses are referring to *virtual* memory
 - We cannot use the linker's default settings
 - We need a special *linker file* with our configuration
- High-level code has requirements
 - Address translation, Runtime environment, Stack, ...
 - Separate startup code has to prepare the execution of high-level language code

- Classification
- Compiling and Linking
- Boot process
- Debugging
- Summary

- Bootstrapping: A simple system starts a more complex system

- Name is derived from *Münchhausen*, who supposedly pulled himself out of a swamp by his on hair



■ BIOS = Basic Input Output System

- Firmware on motherboard → Initializes hardware and allows access to devices
- Provides many functions per software interrupt (only in *Real Mode*)
- Stored on a single chip
 - formerly: ROM, later: EEPROM, today: NAND flash memory
- The term *BIOS* was introduced by the CP/M operating system in 1975

■ Starts with a POST = Power On Self Test

- CPU check → built-in self test of the CPU
- Initialization of chipset & main memory
- Initialization of different mainboard components



BIOS: Real Mode

- Default CPU mode after power-on / reset
- 16-bit register, 64 KiB segments
- Only 20-bit physical addresses (max 1 MiB main memory)
- Effective address = (segment << 4) + offset

Intel 8086



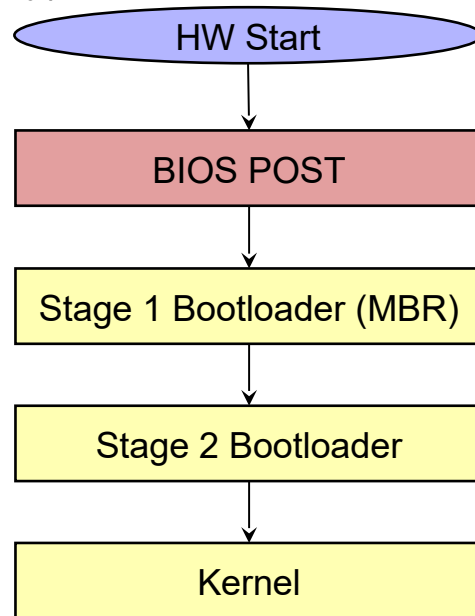
cs	ax		si	ip
	ah	al		
ds	bx		di	flags
	bh	bl		
es	cx		bp	
	ch	cl		
ss	dx		sp	
	dh	dl		

16 Bit Segment	1	2	3	4	
16 Bit Offset		5	6	7	8
20 Bit Address	1	7	9	B	8

- Determine first/next boot device
 - Check for signaute `0x55AA` in sector 0 = Master Boot Record (MBR)
 - In case of wrong signature or read error → next device
- If found, the BIOS loads the first sector to a special memory address and jumps to it
 - First loader (Stage 1) must fit into 512 Byte → Assembler
 - If the loaded code returns → next device
- The loader in the MBR loads the real boot loader (Stage 2) using the BIOS
 - May show a menu interface, if multiple operating systems are installed
 - Loads the kernel image into memory and jumps to it

- Kernel has an entry function (usually written in assembly)

- Switches the CPU to *Protected Mode* (32 Bit)
- Afterward to *Long Mode* (64 Bit)
- Performs basic setup (Stack, 1:1-Paging, etc.)
- Calls a function written in a high-level language (often C)



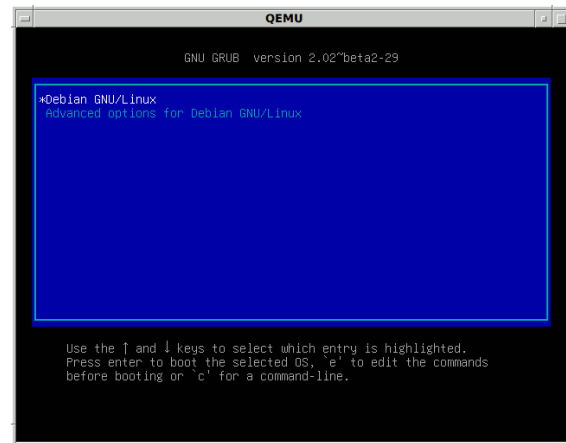
Boot process using GRUB

■ GRUB = GNU GRand Unified Bootloader

- Offers a boot menu and allows defining boot parameters
- Reference implementation of the *Multiboot* protocol
 - <https://www.gnu.org/software/grub/manual/multiboot/multiboot.html>
- Kernel images are ELF files with a Multiboot record at the beginning
- Grub switches to Protected Mode (32-bit), loads the kernel image defined in `grub.cfg` and jumps to it
- If specified by the Multiboot record, GRUB switches to a desired graphic mode and can provide system information



■ GRUB supports classic BIOS and the modern UEFI BIOS



- Unified Extensible Firmware Interface (UEFI)
- Software interface between operating system and hardware
 - 64-bit code instead of 16-bit real mode
 - Device drivers for pre-boot environment (including a network stack)
 - Bootmanager, supports disks with a GUID partition table
 - Filesystem support (FAT32), textual und graphical console
 - Extensions: can be loaded from persistent memory devices
 - Supports pre-boot applications
- Additional information: <http://software.intel.com/en-us/articles/about-uefi/>

- Bootloader developed by Niklas Sombert (HHU, Department: Operating Systems)
 - Written in Rust, as a UEFI application → Only works on modern UEFI BIOS
 - Supports *Multiboot* and *Multiboot2* protocols
 - Kernel images are ELF files with a Multiboot record or some Multiboot2 tags at the beginning
 - Parses the configuration file `towboot.toml` and loads the specified kernel image
 - Can also switch to a desired graphics mode, if specified in the Multiboot/Multiboot2 header
- Open Source: <https://github.com/hhuOS/towboot>
- HeineOS uses Towboot as its bootloader

- Classification
- Compiling and Linking
- Boot process
- Debugging
- Summary

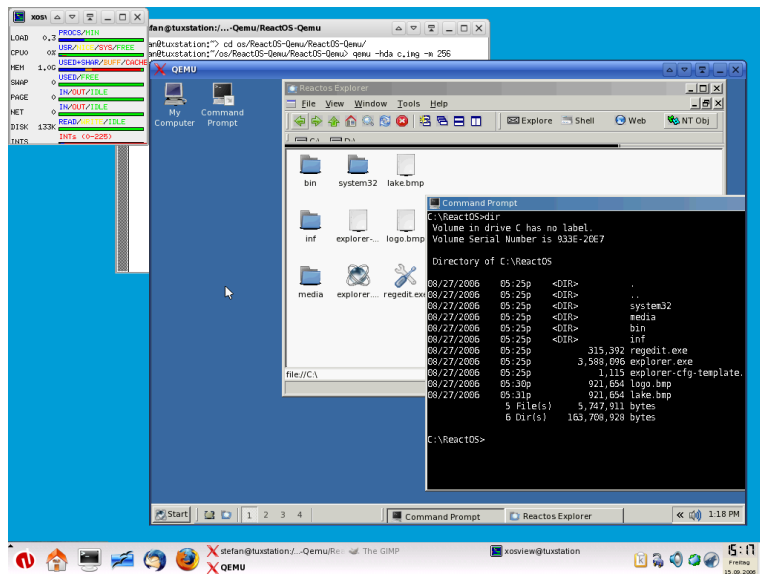
- Not trivial, as `println!()` is part of the Rust runtime environment (not in `core`)
- Furthermore calls to `println!` impact execution speed (slow!)
 - Some errors may not occur anymore after inserting `println!()` calls into the code
 - This is especially true for OS development
 - Oftentimes errors lead to immediate reboot, causing the screen to be erased
- Resorts
 - Serial port
 - PC Speaker
 - Blinking LED

- Emulate specific hardware in software
 - Most emulators support attaching a debugger, to pause the system and inspect variables and CPU registers
 - May also influence system behaviour, but is very helpful nonetheless
- Caution: Emulator and real hardware may differ in details
- *Virtual Machines* and *Emulators* are not the same
 - Example: VMware does not emulate an x86 CPU, but uses a real CPU to directly execute code in the VM

Example: QEMU



- Emulates different CPU architectures, e.g. x86 (also supports multiple cores)
- Emulates a full PC
 - Memory, Devices (even sound and network cards)
- Written in C
- Useful for OS developers
 - Debugger support (GDB)
 - CPU/Memory dump on crash
- Used for HeineOS development



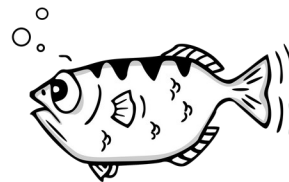
Example: gdb

- gdb = GNU Debugger

- Again: *Hello, World*

```
fn main() {  
    println!("hello world");  
}
```

```
$ rustc -g hello.rs
```



- Notice: Compilers distinguish between *Debug* und *Release* builds
 - The Debug version includes debug information in the program image
 - For example names of parameters or local variables, etc.
 - Release code is usually much faster (compiler optimizations)

- INT3 triggers a *breakpoint interrupt* (TRAP)
 - Is placed inside the code specifically by the user level debugger (e.g., gdb)
 - TRAP handler inside the kernel moves control flow to the debugger
- Setting *Trap Flags (TF)* inside the status register (RFLAGS) triggers a *debug interrupt (INT 1)* after every instruction
 - Useful for implementing a single step mode in the debugger
- The debug registers *DR0-DR7* (introduced with the i386) can monitor up to 4 breakpoints without manipulating code
 - Substantial advantage if code is stored in ROM/FLASH or non-writable memory segments
 - Used by kernel level debuggers (e.g., kdb)
 - Only accessible in Ring 0

Debug registers in x86

Breakpoint Registers

Breakpoint 0: Linear Address
Breakpoint 1: Linear Address
Breakpoint 2: Linear Address
Breakpoint 3: Linear Address
Reserved
Reserved

Debug Status Register

	B	B	B		B	B	B	B
	T	S	D		3	2	1	0
31				16	15			0

Debug Control Register

LEN	RW	LEN	RW	LEN	RW	LEN	RW	-	-	G	-	-	-	-	G	L	G	L	G	L	G	L
3	3	2	2	1	1	0	0			D					3	3	2	2	1	1	0	0
31								16	15													0

Breakpoint 0: Linear Address	
Breakpoint 1: Linear Address	
Breakpoint 2: Linear Address	
Breakpoint 3: Linear Address	
Contains information about the TRAP cause	

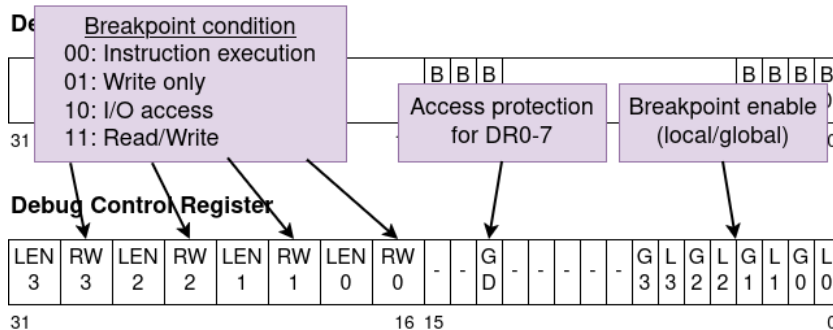
																B	B	B																	B	B	B																																																																																																																																																																																																																																										
																T	S	D																	3	2	1																																																																																																																																																																																																																																										
31																16																15																14																13																12																11																10																9																8																7																6																5																4																3																2																1															

Debug Control Register										Single-Step				Breakpoint 0-3								
LEN	RW	LEN	RW	LEN	RW	0	0	-	-	D	-	-	-	-	G	3	2	2	1	1	0	0
						16				15												

Debug registers in x86

Breakpoint Registers

Breakpoint 0: Linear Address
Breakpoint 1: Linear Address
Breakpoint 2: Linear Address
Breakpoint 3: Linear Address
Reserved
Reserved



- Allows debugging on platforms, that do not offer an interactive user interface
 - Requires a communication connection (Serial port, Ethernet, etc.)
 - A device driver for the communication device is needed
 - Target platform may also be an emulator (QEMU → TCP)
- The debugging component on the target platform should be as simple as possible



■ Communication protocol: *GDB Remote Serial Protocol* (RSP)

- Defines the communication interface (GDB *stub*)
- Only uses ASCII strings
- Message format: `$<Command/Response>#<Checksum>`
- Messages are immediately answered with + (OK) or - (Error)

■ Examples:

- `$g#67` → Read all register contents
 - Answer: + `$123456789abcdef0...#...` → register 1 is 0x12345678, register 2 is 0x9ab...
- `$G123456789abcdef0...#...` → Set all register contents
 - Answer: + `$OK#9a` → Success
- `$m4015bc,2#5a` → Read 2 byte at memory address 0x4015bc
 - Answer: + `$2f86#06` → Value is 0x2f86

Remote debugging using QEMU

```
~/Lehre/HeineOS lesson-1   
cargo make --no-workspace debug
```

```
QEMU [Angehalten]
Maschine Ansicht

[ INFO]: towboot/src/boot/video.rs@096: set (800, 600) as the video mode
[ INFO]: towboot/src/main.rs@109: booting HeineOS...
```

```
~/Lehre/HeineOS lesson-1   
> gdb
GNU gdb (GDB) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) file loader/kernel.elf
Reading symbols from loader/kernel.elf...
⚠ warning: Missing auto-load script at offset 0 in section .debug_gdb_scripts
of file /home/fabian/Lehre/HeineOS/loader/kernel.elf.
Use 'info auto-load python-scripts [REGEXP]' to list them.
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x0000000000000000 in ?? ()
(gdb) break *main
Breakpoint 1 at 0x1900500: file kernel/src/boot.rs, line 45.
(gdb) continue
Continuing.

Breakpoint 1, kernel::main (multiboot_magic=0, multiboot=0x0) at kernel/src/boot.rs:45
45     pub extern "C" fn main(multiboot_magic: u32, multiboot: &multiboot::BootInfo) -> ! {
(gdb) next
48     if log::set_logger(&LOGGER).is_err() {
(gdb) next
51     log::set_max_level(log::LevelFilter::Debug);
(gdb) next
55     if multiboot_magic != multiboot::MULTIBOOT2_MAGIC {
(gdb) next
62         .find_tag:::<multiboot::FramebufferInfo>(multiboot::TagType::FramebufferInfo)
(gdb) □
```

- Classification
- Compiling and Linking
- Boot process
- Debugging
- Summary

- OS development differs significantly from application development
 - (Most) libraries are missing
 - Foundation is formed by the *naked* hardware
- First steps are hard
 - Compiling & Linking
 - Boot process
 - System initializatzion
- Debugging requires infrastructure
 - Display driver für `print!`-Debugging
 - GDB *stub* and communication device (and driver) for remote debugging
 - Hardware debugging support